

Neural Network Project
CMPS3500

Group 5

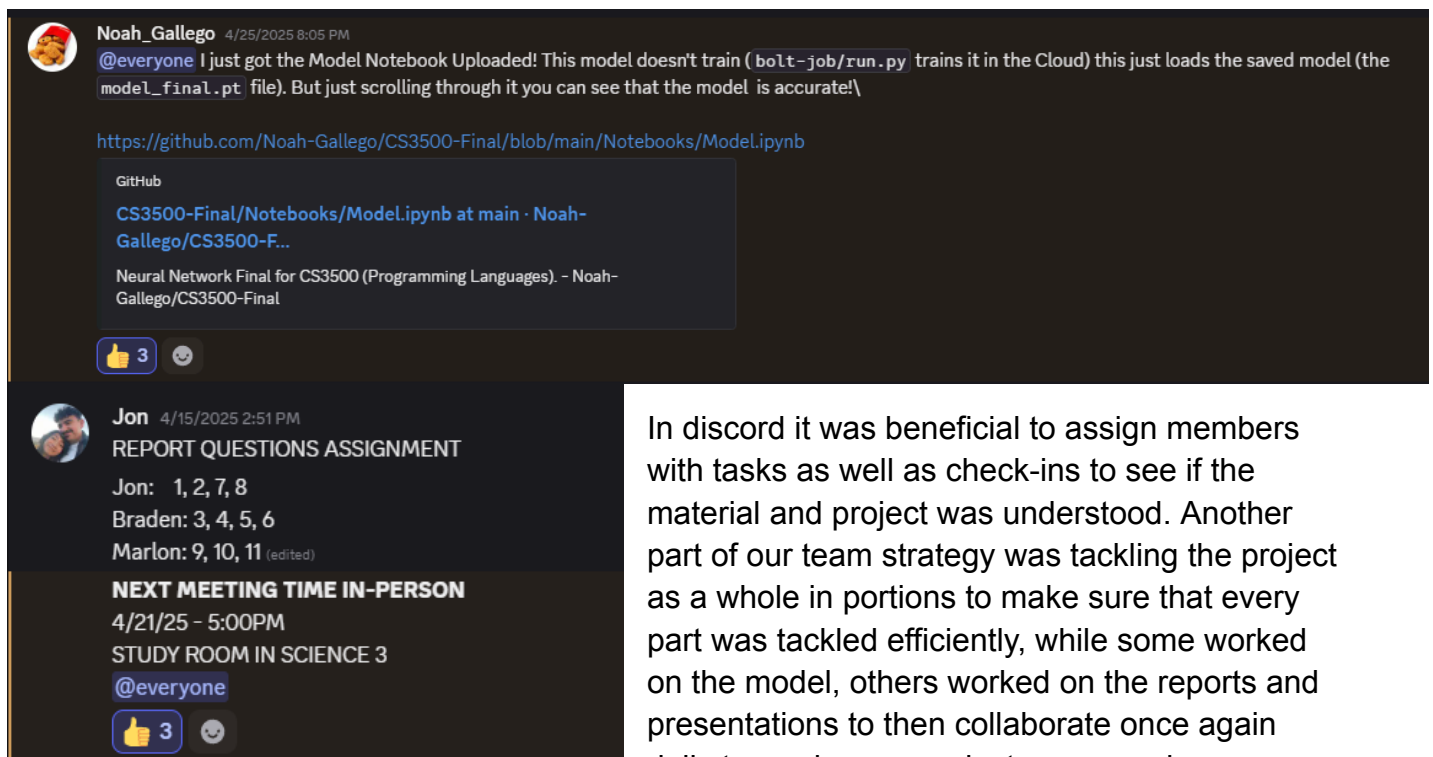
Noah Gallego
Braden Smelser
Marlon Elvir
Jonathan Torres

Team Strategy

When thinking about strategy in a team it was very important for us to fully understand the concepts of Neural Networking and Jupyter Notebook. First and foremost our strategy was for the group to understand the project goals, and fundamentals. As a team we scheduled regular meetings both in person and online, for our in person meetings we had one person in our group with the most knowledge of Neural Networks be the guide of the group to catch everyone up to speed so we could be as efficient as possible.

Our meetings on discord consisted of assigning roles and check in, as we all discussed problems we faced and offered any solutions to those problems that could be experienced. On discord it made it easier to collaborate when a lot of team members had conflicting schedules, making it a very simple solution to collaborate from everywhere.

In our group we thought that a synchronized effort was very important, early on in the notebook finalization we made sure to make a github that was our final piece of the project, this is where the overall group decided final checks were made and pushed. Individually we made strides and fused them into one final push that was agreed upon on the github. This style was done for both our notebook and our menus.



The screenshot shows two messages in a Discord chat. The first message is from Noah_Gallego, dated 4/25/2025 8:05 PM, announcing the upload of a Model Notebook to GitHub. It includes a link to the repository and a preview of the GitHub page. The second message is from Jon, dated 4/15/2025 2:51 PM, reporting on assignment questions and providing the next meeting details.

Noah_Gallego 4/25/2025 8:05 PM
@everyone I just got the Model Notebook Uploaded! This model doesn't train (`bolt-job/run.py` trains it in the Cloud) this just loads the saved model (the `model_final.pt` file). But just scrolling through it you can see that the model is accurate!\

<https://github.com/Noah-Gallego/CS3500-Final/blob/main/Notebooks/Model.ipynb>

GitHub
CS3500-Final/Notebooks/Model.ipynb at main · Noah-Gallego/CS3500-F...
Neural Network Final for CS3500 (Programming Languages) - Noah-Gallego/CS3500-Final

Jon 4/15/2025 2:51 PM
REPORT QUESTIONS ASSIGNMENT
Jon: 1, 2, 7, 8
Braden: 3, 4, 5, 6
Marlon: 9, 10, 11 (edited)

NEXT MEETING TIME IN-PERSON
4/21/25 - 5:00PM
STUDY ROOM IN SCIENCE 3
@everyone

In discord it was beneficial to assign members with tasks as well as check-ins to see if the material and project was understood. Another part of our team strategy was tackling the project as a whole in portions to make sure that every part was tackled efficiently, while some worked on the model, others worked on the reports and presentations to then collaborate once again daily to produce a product we agreed upon.

Data Handling Implementation

Extracting crucial data from a saturated dataset took a series of steps which were performed in a specific order.

Our goal:

Upon obtaining the dataset featuring 21 columns and thousands of rows of Los Angeles crime data, we needed to clean and organize the data. This consisted of dropping rows that weren't needed for classifying crime status. When a neural network "learns" it uses specific data imputed by the programmer, this data is fed into artificial neurons or nodes and based upon the activation function the model will begin to predict outcomes. This is why we only need crucial, relevant data to incur proper results.

Cleaning:

To tackle this task we began to study and group the data in clusters based on relevancy. Once we did this we were able to pick what data could be tossed because it serves no purpose. We dropped: null values, repeated rows, unnecessary outliers, and irrelevant columns. The reason for these removals is because these datasets are unnecessary and repetitive, they don't contribute to the success of our neural network. Dropping these rows enabled us to use a clean organized dataset for our model, allowing it to use only applicable data for training.

Incorporation:

Once our data was cleaned we fed it into our neural network, ready for training and testing. We noticed mo-codes were unnecessary and dropped these values with their corresponding rows. By using built-in PyTorch and Pandas functions we visualized and extracted the data for the model. We were able to extract data types and every detail about the information we were going to train, being able to see a breakdown of rows and columns. Seen below:

In [20]:

```
train.info() # ALL datatypes float, int, boolean --> We are now able to move on
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 189451 entries, 0 to 189450  
Columns: 604 entries, DR_NO to Status  
dtypes: bool(563), float64(2), int64(39)  
memory usage: 161.0 MB
```

Structure/Architecture & Error Handling

To predict arrest outcomes from Crime and Incident data, we implemented two neural networks models: A Multi-Layer Perceptron and a TabTransformer. Both were built using PyTorch, a widely adopted learning framework. The MLP serves as a baseline model with three fully connected layers of sizes 256, 128, and 1 respectively using SELU activations and AlphaDropout to promote self-normalization and mitigate overfitting. It outputs a single logit, which is later transformed into a probability via a sigmoid function. The TabTransformer adapts the transformer encoder for tabular data by projecting input features into a 128-Dimensional embedding space, followed by two TransformerEncoder layers with four attention heads and GELU activation. Its output is passed through a dense head with ReLU and throughout, producing a final binary score. Both models are trained using a custom Focal Loss function to address class imbalance by emphasizing harder-to-classify examples. During inference, their sigmoid outputs are averaged in an ensemble to improve predictive performance. This dual-model approach enables a comparison between a traditional MLP and a more expressive attention-based architecture in handling real-world, imbalance classification tasks.

Robust error handling was implemented to ensure system stability during data processing, training, and inference. The Menu uses try-except block to catch and report errors with clear messages, while boolean flags enforce correct use and execution order. User prompts safeguard against unintended resource-intensive operations, and non-critical warnings are suppressed to improve usability. To support portability, the system searches multiple paths for model files, enhancing resilience across environments. Collectively, these measures promote fault tolerance and a smooth use experience.

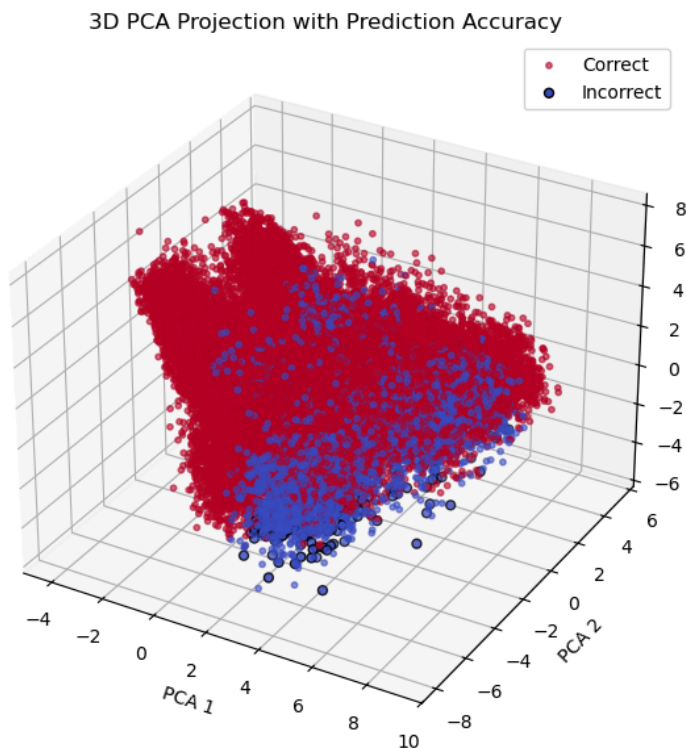
Consistent and descriptive naming was used throughout the system to enhance readability, maintainability, and clarity. Model classes like MLPModel and TabTransformer follow PyTorch conventions and are reused consistently. Data columns from external sources are converted to snake_case during preprocessing, and engineered features maintain this convention for uniformity. Global variables and model files also follow structured naming to support reproducibility and modular development. This disciplined approach facilitates collaboration and future scalability.

The codebase uses clear, structured comments to enhance readability and maintainability. Section headers divide the script into logical blocks, while inline comments explain key operations, especially in domain-specific preprocessing. User prompts and safeguards guide interactions during intensive tasks, reducing errors. Detailed comments in complex functions clarify feature engineering steps such as time categorization and holiday calculations. These practices result in a well-documented and accessible codebase, supporting collaboration and future scalability.

Our Neural Network

We implemented a binary classification model using PyTorch to predict the status of a crime (Arrest vs. No Arrest). A custom Dataset class efficiently converts processed data into PyTorch tensors for batch processing during training and evaluation. The final layer uses a sigmoid activation function to output probabilities for binary classification.

For individual evaluations we use a visualized 3D Principal Component Analysis (PCA) Projection that is all processed from classified metrics. Data is then shown through a scatter plot that shows us the accuracy and inaccuracy between predictions. Through this plot we are able to properly see the errors our system makes and adjust them if needed, all in all contributes to our comprehensive binary classifier for crime status prediction.



During training, we used several key hyperparameters: a learning rate of 0.001 (that is scheduled), batch size of 1024, and 500 epochs, with an early stopping patience of 15. We applied PowerTransformer scaling and SMOTE oversampling to address class imbalance. The model also used SELU activation, AlphaDropout (0.1), and a weight decay of $1e-4$ to support generalization.

These design choices support both performance and interpretability, contributing to a robust pipeline for crime status prediction.

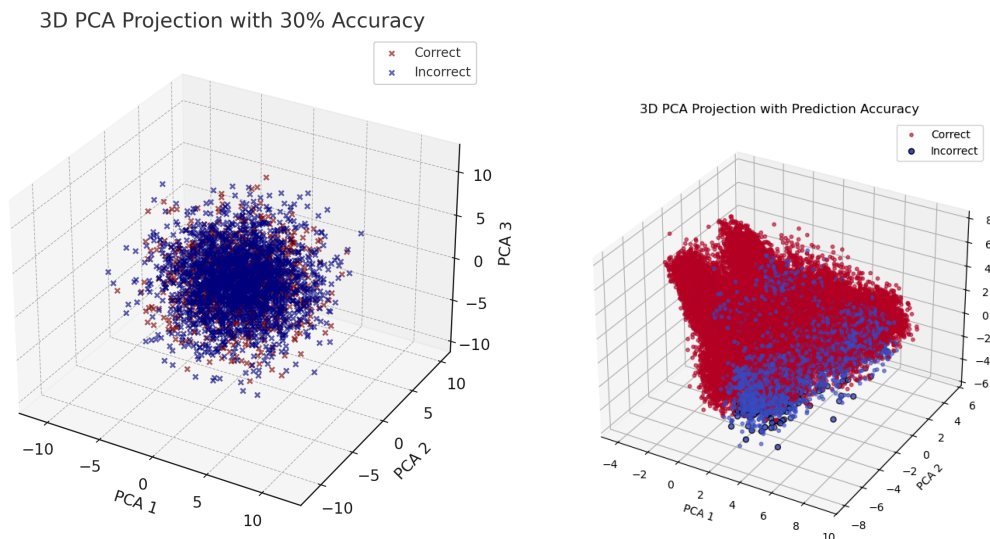
Testing Our Model

When we first tested our model, it performed at around 30% accuracy, which was far below expectations. This early result highlighted the complexity of working with neural networks and the steep learning curve we faced. Through trial and error, we managed to improve the accuracy to about 50%, but we knew that wasn't enough.

One major challenge was achieving consistent results. We noticed that the model sometimes performed well and other times poorly, depending on how the data was handled. We suspected the model was overly familiar with static, hardcoded data—leading to overfitting and poor generalization.

To address this, we restructured our pipeline to allow flexible data loading and ensured each run used clean, properly processed data. We also refined our hyperparameters and added proper evaluation procedures.

After these changes, we saw significant improvement, with test accuracy ranging from 75% to 90% across multiple runs. While achieving consistent high performance remained a challenge, these results marked a major breakthrough for our team and demonstrated the model's real potential.



Conclusion

With the learning curve most group members had to face, we believe we achieved a great benchmark when it comes to neural networks. From the start of the project to now there have been substantial improvements in this field. Upon implementing our model we first sat at around 50% accuracy, now we are sitting at 75% to 90% which is a success as stated before. This is thanks to learning about activation functions, encoding, and back propagation. These 3 categories proved to be very challenging concepts but were essential in building our model.

One of the biggest challenges we faced was getting the model to recognize the minority class versus the majority class. The data imbalance showed that the model wasn't learning class zero but was primarily learning class 1. This is a common issue developers have when building and testing their models. To fix this issue, we implemented class weighting, a focal loss, and SMOTE (upsampling) which means we created fake samples of class 0 and brought it up to the same level as class 1. Upon solving this issue our model performed way better and gave us our final accuracy.

From tackling the data cleaning portion to building a fully functional neural network, we achieved a monumental task in a short period of time. Our model isn't perfect but it is efficient and accurate, it has predicted thousands of crimes given the test data we fed it. This required us to work as a team and collaborate as engineers who have never worked with one another. We hope to use these skills; teamwork and coding, in our future careers and classes along with the understanding of this huge breakthrough in programming.